



eBook

A Compact Guide to Retrieval Augmented Generation (RAG)

2nd Edition

Definitions, components and basics for practitioners



Contents

Introduction: Your Pain Points, Our Starting Point3

What is Retrieval-Augmented Generation? 4

 What RAG Is (and isn't) 5

 The RAG Advantage 6

 How RAG fits with other modeling methods 6

RAG Components..... 7

 1. Data pipeline7

 2. RAG chain (retrieval → augmentation → generation) 8

 3. Evaluation and monitoring 9

 4. Governance and LLMOps 9

The RAG Pipeline End-to-End 10

 Ingestion 10

 LLMs and prompts13

 Document processing14

 Vector Search and embedding models17

 Indexing 19

 Retrieval..... 20

 Reranking.....21

 Context assembly..... 22

 Generation..... 22

 Evaluation and monitoring..... 23

 Current best practice: Hybrid retrieval plus reranking.....27

 Hybrid search implementation 28

 Beyond basic RAG: Instruction-aware retrieval.....30

 RAG on Databricks.....32

 Agent Bricks: Automated agent development.....32

Summary33

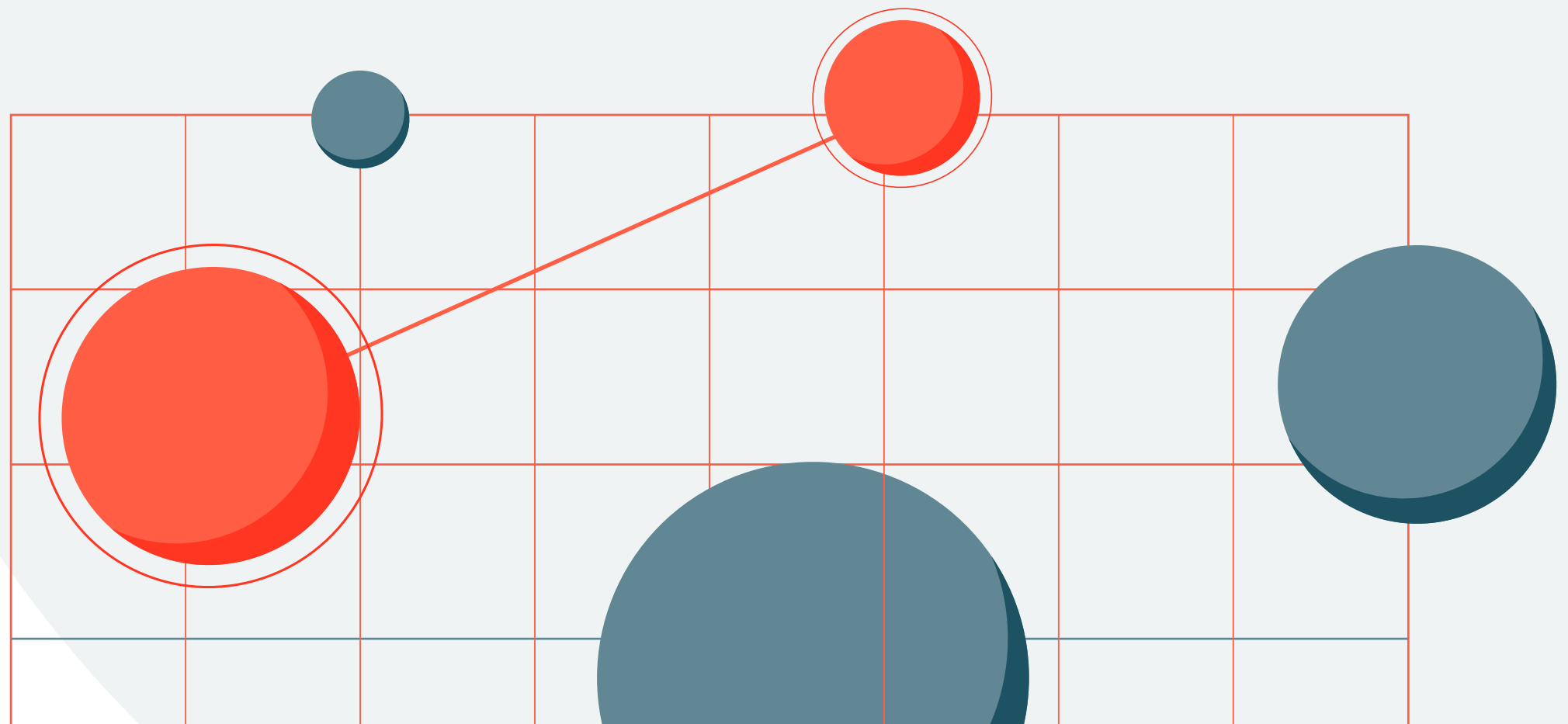
 GenAI training 33

 Additional resources 33

Introduction

Your Pain Points, Our Starting Point

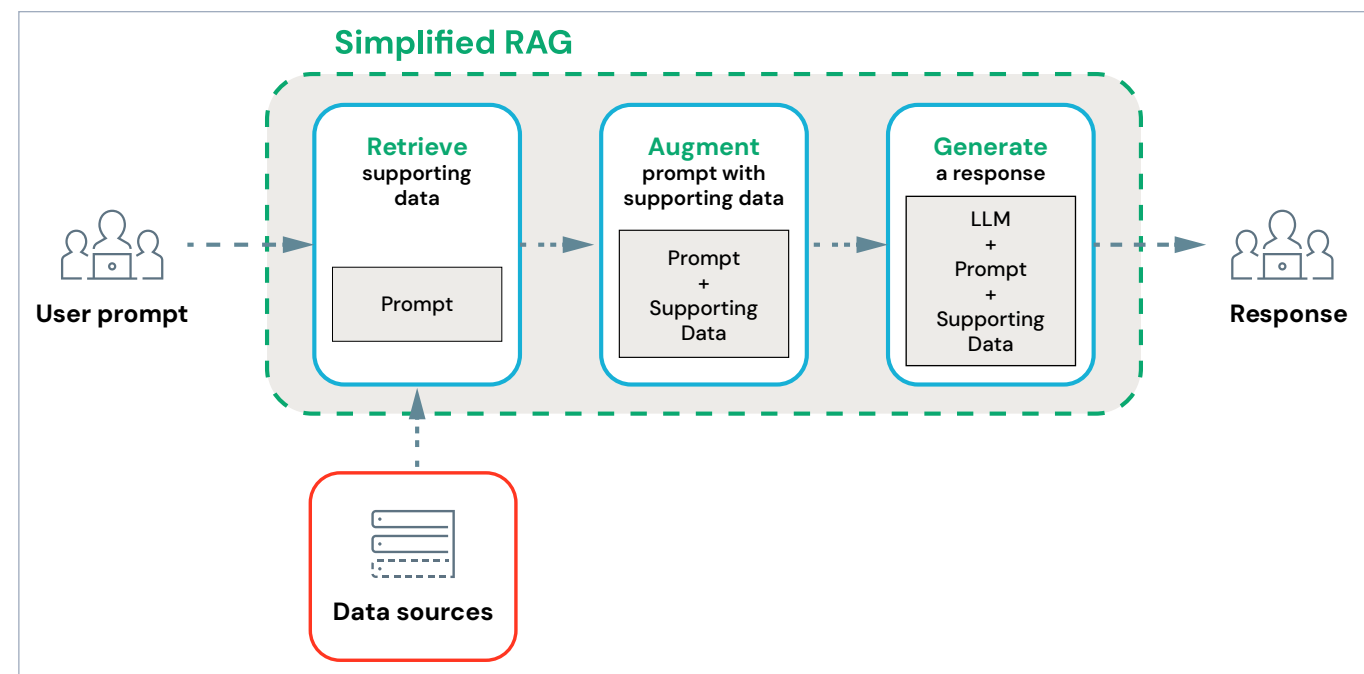
Retrieval augmented generation (RAG) is a powerful technique that combines large language models (LLMs) with real-time data retrieval to generate more accurate, up-to-date and contextually relevant responses. This approach is especially valuable for answering questions about proprietary, frequently changing or domain-specific information.



What is Retrieval Augmented Generation?

In its simplest form, a RAG agent performs three core steps:

1. **Retrieval:** The user request queries an external knowledge base (vector store, keyword search or SQL database) to retrieve supporting data for the LLM's response
2. **Augmentation:** The supporting data is combined with the user request, often using a template with additional formatting and instructions, to create an enriched prompt
3. **Generation:** The augmented prompt is passed to the LLM to generate a response to the user request



This guide focuses on RAG done right: hybrid retrieval instead of single-mode search, reranking to prioritize the best evidence, ingestion that supports multiple formats and evaluation that directly measures retrieval quality and groundedness. It also shows how RAG fits into agentic systems, where retrieval is one tool among many but still determines whether the agent can reason over the right context.

What RAG is (and isn't)

Retrieval augmented generation is like giving your AI a personalized research assistant. When a user asks a question, RAG fetches relevant information in real time and feeds it to the language model along with the original query, creating a knowledge-enhanced prompt that helps the model answer with information it never encountered during training.

RAG is *not*:

- **Model training:** Unlike fine-tuning or pretraining, which permanently modify how a model thinks, RAG keeps the model unchanged but supplements it with fresh context for each query
- **Generic web search:** While search engines crawl the public internet for broadly relevant results, RAG focuses on your private information ecosystem — documents, tickets, code, logs — with strict security, traceability and compliance controls
- **A complete agent system:** Agents coordinate multiple tools, including retrievers, databases, code execution and external APIs. While many agentic systems utilize RAG, it's just one tool in this toolkit, albeit a crucial one for knowledge-intensive tasks.

RAG shines when:

- Your knowledge base evolves regularly (documentation, policies, product versions)
- You have more content than can fit in a single prompt
- You need traceable answers with proper citations to source material

When these conditions apply, RAG provides a controlled way to ground the model in facts while leveraging its reasoning capabilities.

The RAG advantage

- **Unlock proprietary knowledge:** Incorporate company-specific information like internal documents, emails and memos that weren't in the model's training data
- **Stay current:** Supply the model with the latest information from continuously updated knowledge bases
- **Build trust through citations:** Enable the model to reference specific sources, making responses verifiable
- **Maintain security:** Apply access controls to ensure users only receive information they're authorized to see

How RAG fits with other modeling methods

When designing an AI system, you typically have four main levers:

1. **Prompt engineering:** Steer behavior with instructions, examples and formatting
2. **Fine-tuning:** Adapt a model to your domain or tasks using labeled examples
3. **Pretraining:** Train on large, unlabeled corpora to change the model's base knowledge and style
4. **RAG:** Keep the model fixed and give it fresh, task-specific context at query time

Prompt engineering is fast and cheap, but it's limited when the model's underlying knowledge or behavior diverges from your needs. Fine-tuning and pretraining can deeply align behavior but are more expensive and do not solve fast-changing or access-controlled knowledge problems.

RAG does not replace these methods — it complements them by solving the “I need this answer to be grounded in my data now” requirement.

On the Databricks Platform, you can combine these levers in one platform: use foundation models and fine-tuning where needed, and apply RAG with Databricks Vector Search and pipelines over your governed data.

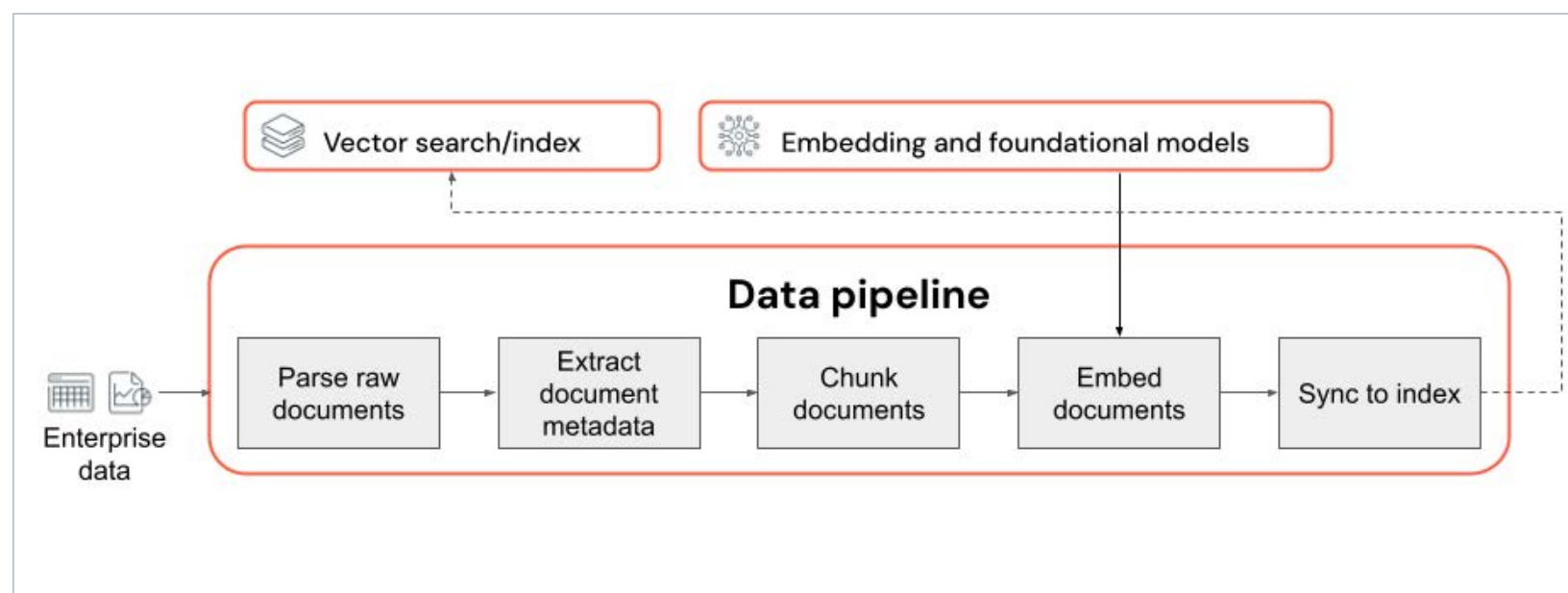
In practice, mature systems often use combinations — for example, a domain-tuned model accessed via RAG over internal data, with prompts optimized for grounded answers. This guide focuses on the RAG part of that picture.

RAG Components

A typical RAG application involves several core components.

Data pipeline

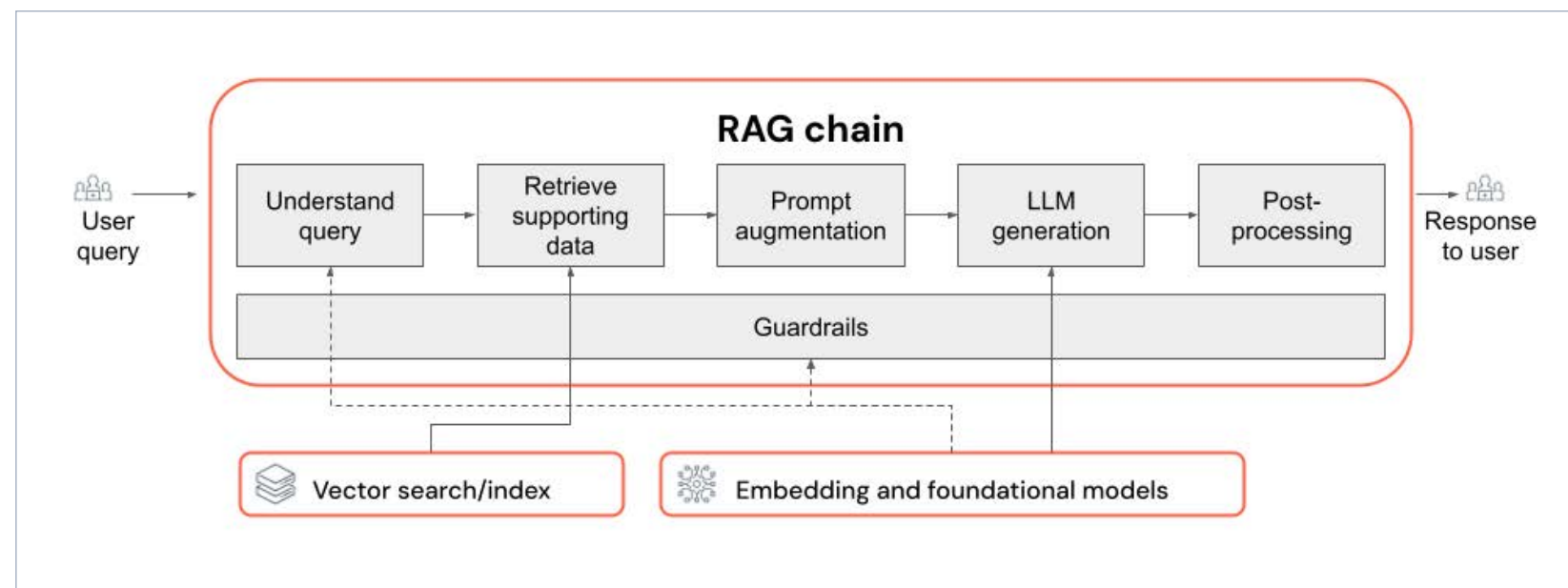
Preprocess and index documents, tables and other data sources for fast, accurate retrieval. This includes cleaning, chunking, extracting metadata and building the underlying search indexes.



RAG chain (retrieval → augmentation → generation)

A sequence of steps that:

1. Interprets the user's question and any system instructions
2. Retrieves supporting data from the index
3. Augments the model prompt with the supporting data and relevant metadata
4. Generates a response from an LLM using the augmented prompt

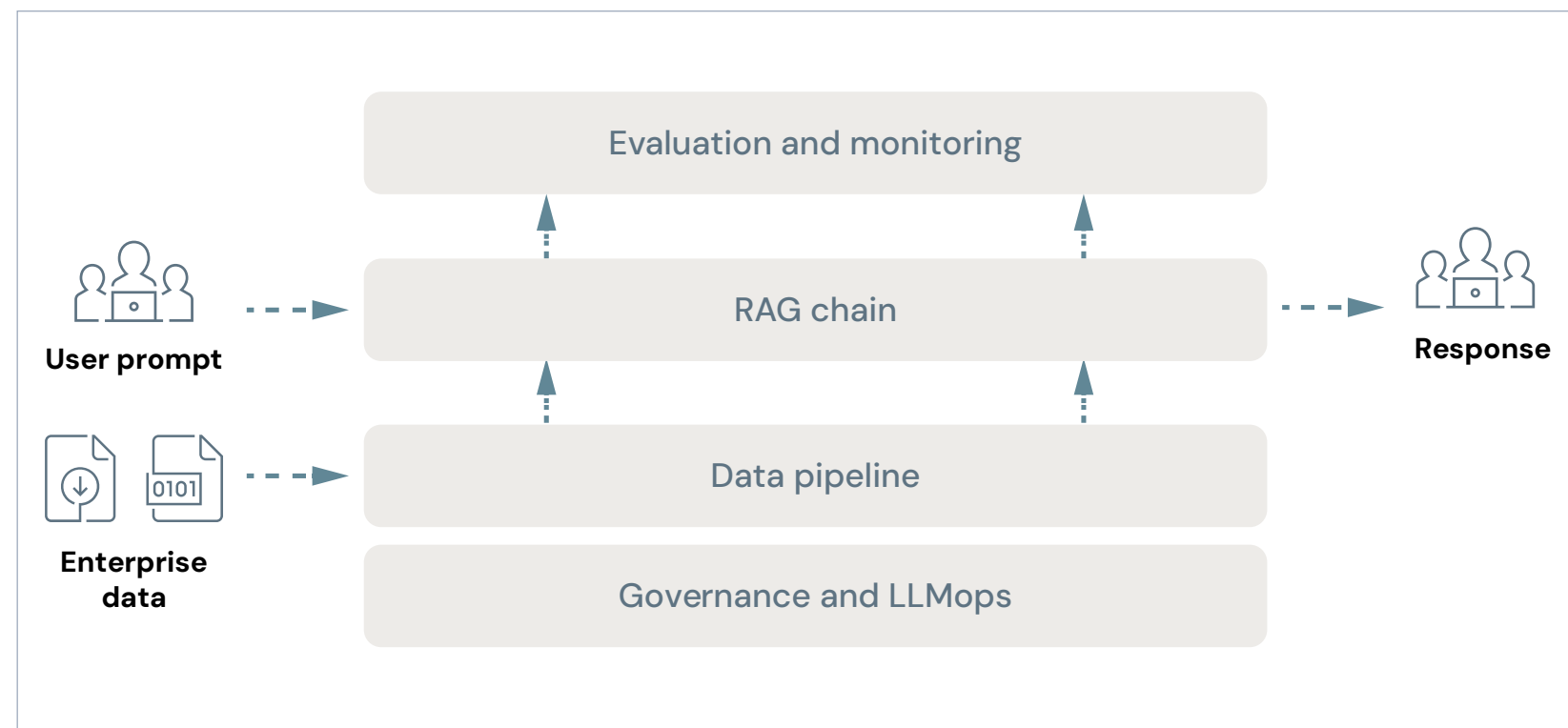


Evaluation and monitoring

Continuously assess the RAG application for quality, cost and latency. Track retrieval quality (e.g., recall@k), answer correctness and groundedness, and user feedback to ensure the system meets business requirements.

Governance and LLMOps

Manage the lifecycle of data, models and chains. Maintain data lineage, enforce access controls and permissions, and log decisions so you can audit behavior, roll out changes safely and comply with internal and external policies.



The RAG Pipeline End-to-End

A production RAG system is more than “embed some text and call the model.” Think of the pipeline as:

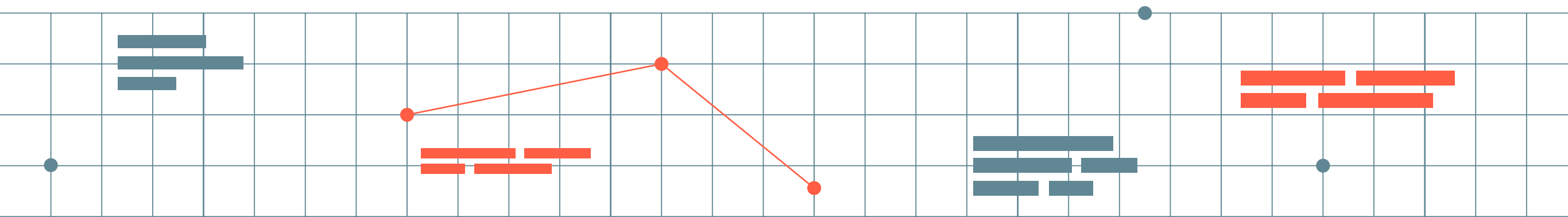
Ingestion → Indexing → Retrieval → Reranking → Context assembly → Generation → Evaluation → Feedback loop

Ingestion

Ingestion collects raw sources — PDFs, wikis, tickets, chat logs, code, tables, images — and turns them into clean, structured assets. This includes:

- Extracting text
- Preserving headings and section boundaries
- Capturing metadata (source, author, timestamps)
- Normalizing formats

For documents mixing text and images, extract captions and maintain links between figures and surrounding paragraphs.



DATA TYPES FOR RAG

Unstructured data (data without specific schema):

- PDFs
- Google/Office documents
- Wikis
- Images
- Videos

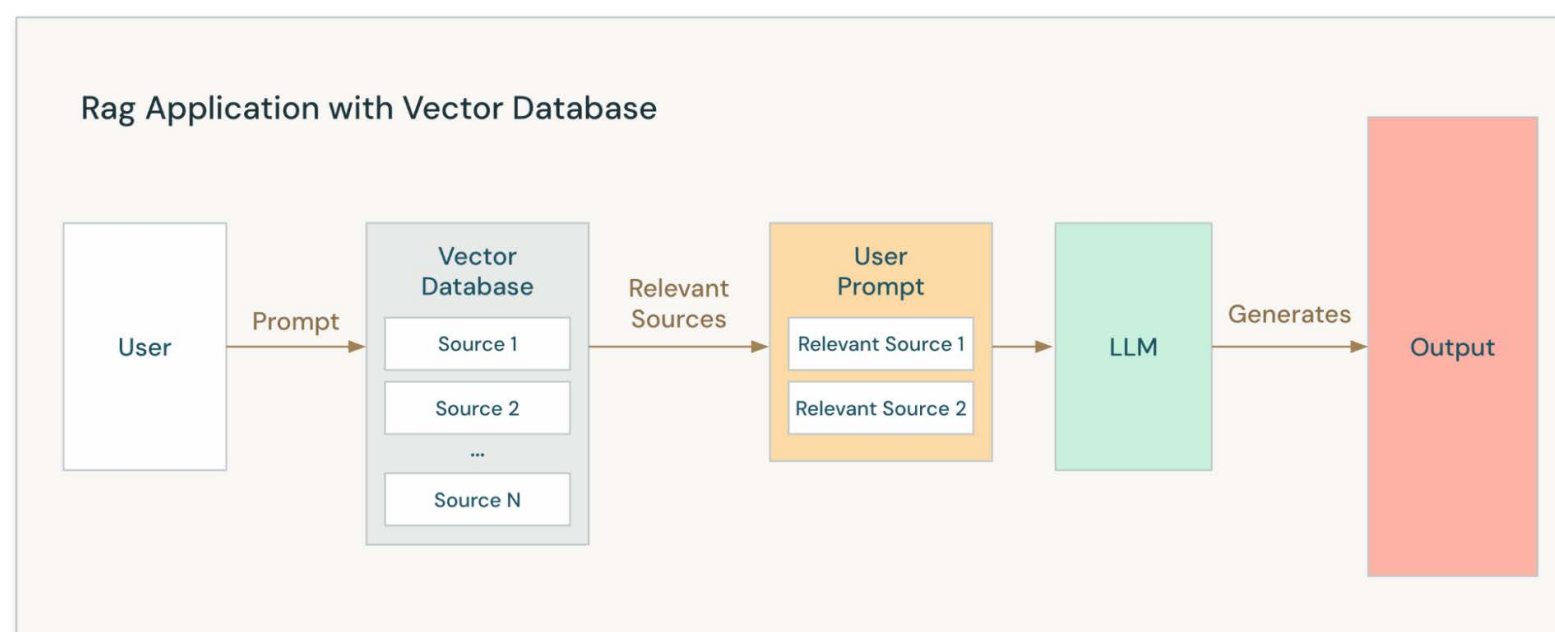
Structured data (tabular data with specific schema):

- Customer records in BI or data warehouse systems
- Transaction data from SQL databases
- Data from application APIs (e.g., SAP, Salesforce)

RAG gives the LLM all the information it needs to answer questions, enables it to provide answers about topics it was not trained on and reduces the likelihood of hallucinations.

For example, an LLM trained on public data can't answer questions about a company's internal memos or project documents — it's likely to hallucinate if asked. A RAG application can supply all or parts of those documents to the LLM, giving it the context it needs to provide correct and relevant answers.

This guide focuses on RAG over stored unstructured data such as PDFs, scraped web pages or code, using Vector Search for retrieval.



LLMs and prompts

LLMs are a class of machine learning algorithms that can interpret, manipulate and generate text-based content. They're trained on massive text datasets from diverse sources, including books, internet text and code repositories.

During training, the model learns statistical relationships between words and phrases, enabling it to generate new text using the context of text it has already seen or generated.

LLMs are typically used via prompting. A prompt is text that a user provides to an LLM, and the LLM responds to it. Prompts can take many forms:

- **Text completion:** "Jack and Jill went up the hill to..."
- **Questions or instructions:** "What happened to Jack after Jack and Jill went up the hill?"

RAG applications that enable users to ask questions about text generally use instruction-following and question-answering LLMs.

LLMs can typically handle prompts of at least several paragraphs in length. This is important for RAG, where the user's question is combined with information retrieved from an external data source, forming the new, augmented prompt.

Document processing

PARSING INPUT DOCUMENTS

Raw documents may not be in a format suitable for RAG with Vector Search. Processing steps include:

- Converting images to text
- Processing tables or images
- Cleaning up or removing extraneous text (page headers, page numbers)

Parse raw input documents and convert them into a format — usually text — that works with the rest of the RAG pipeline.

SPLITTING DOCUMENTS INTO CHUNKS

You typically don't want to retrieve entire books, web pages or articles in a RAG application. Instead, split documents into smaller chunks so you can send more specific results to the LLM for context.

Chunk size affects output quality:

- **Too small:** May not include enough context to address the user's query
- **Too large:** The LLM may fail to pull out relevant details, focusing instead on other details in the chunk

There's no one-size-fits-all solution for chunk size. It depends on the source documents, the LLM and the RAG application's goals. Experimentation is essential.

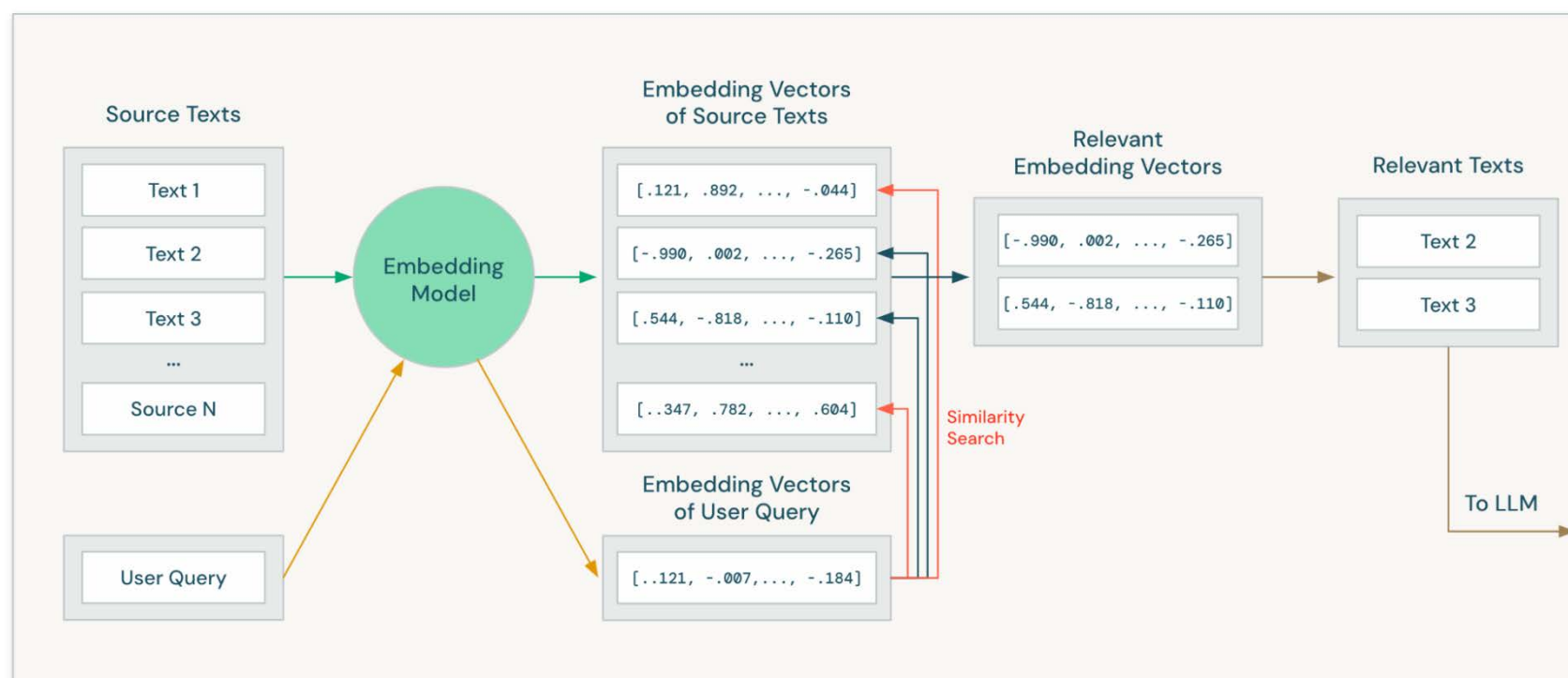
EMBEDDING TEXT CHUNKS

After splitting source documents into manageable chunks, use an embedding model to convert each chunk into a high-dimensional numerical vector.

An embedding model is a special language model that generates a numeric vector (called an *embedding*) from text. Embeddings encode the nuanced and context-specific meaning of each text in numeric form. For example, a good embedding model will know that “raining cats and dogs” is a phrase about weather, not pets.

The power of embeddings for RAG is that they can be mathematically compared to each other. We can measure how numerically similar two embeddings are, which equates to how closely the meanings of their original texts are related.

This is especially useful when we embed a user’s prompt, compare it to embedded texts in the vector database and identify those relevant texts most likely to help the LLM provide a useful answer.



STORING AND INDEXING EMBEDDINGS

Embeddings are stored in a vector database, a specialized database designed to efficiently store and search vector data such as embeddings.

Vector database vs. vector store: These terms are often used interchangeably, though “vector store” can refer to any type of vector storage solution, not just databases. Vector databases often incorporate update mechanisms so newly added chunks can be searched and retrieved immediately.

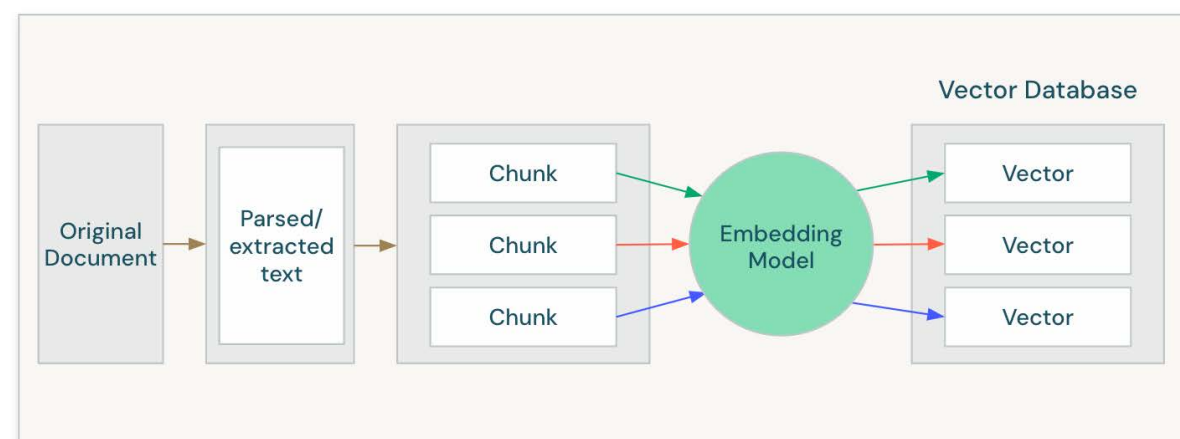
To maintain performance with large numbers of text chunks, embeddings are organized using a vector index — a mechanism that uses various algorithms to organize and map vector embeddings in a way that optimizes search efficiency.

RECORDING METADATA

Capturing metadata along with text chunks allows you to:

- Filter results based on metadata
- Provide detailed references along with results

A RAG application with metadata could provide specific URLs or page numbers for sources retrieved, or allow users to filter by date or source.



Vector Search and embedding models

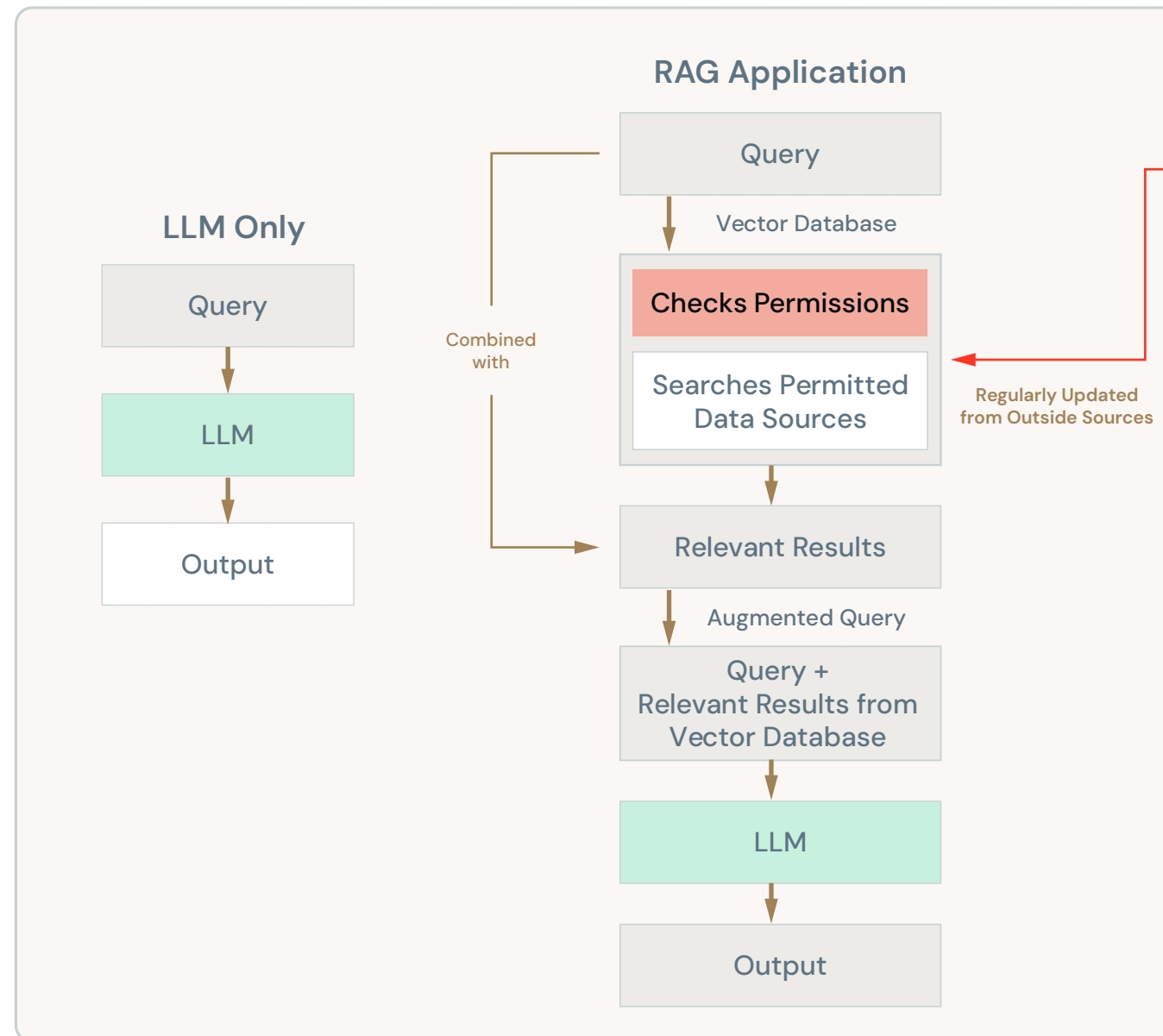
An effective RAG application must find information relevant to the user's prompt and supply it to the LLM. Vector Search addresses the challenge of selecting the most relevant text from potentially millions of documents.

In a RAG system with Vector Search:

1. An embedding model translates each searchable text into a numeric vector that encapsulates the text's meaning
2. The same model converts the user's query to a comparable vector
3. The system mathematically compares the user's query to the texts and identifies those that are most similar and relevant

Crucially, these vectors represent the meanings of the text from which they are generated. When we search for relevant vectors, we're searching on the basis of meaning. This enables us to retrieve the text most relevant to the user's query and pass it to the LLM along with the user's original query.

Important note: Meaning is captured according to the model. Models are trained to capture meanings and relationships that humans care about, but the meanings captured may not be exactly what we want. This is one reason why it's essential to test and evaluate every component of a RAG application.



Indexing

Indexing prepares content for efficient retrieval by:

1. Splitting content into chunks large enough to be meaningful but small enough to be specific
2. Attaching metadata such as document IDs, section titles, timestamps, access controls and content types
3. Computing embeddings for each chunk
4. Building indexes for both sparse (keyword/BM25) and dense (vector) search to enable hybrid retrieval

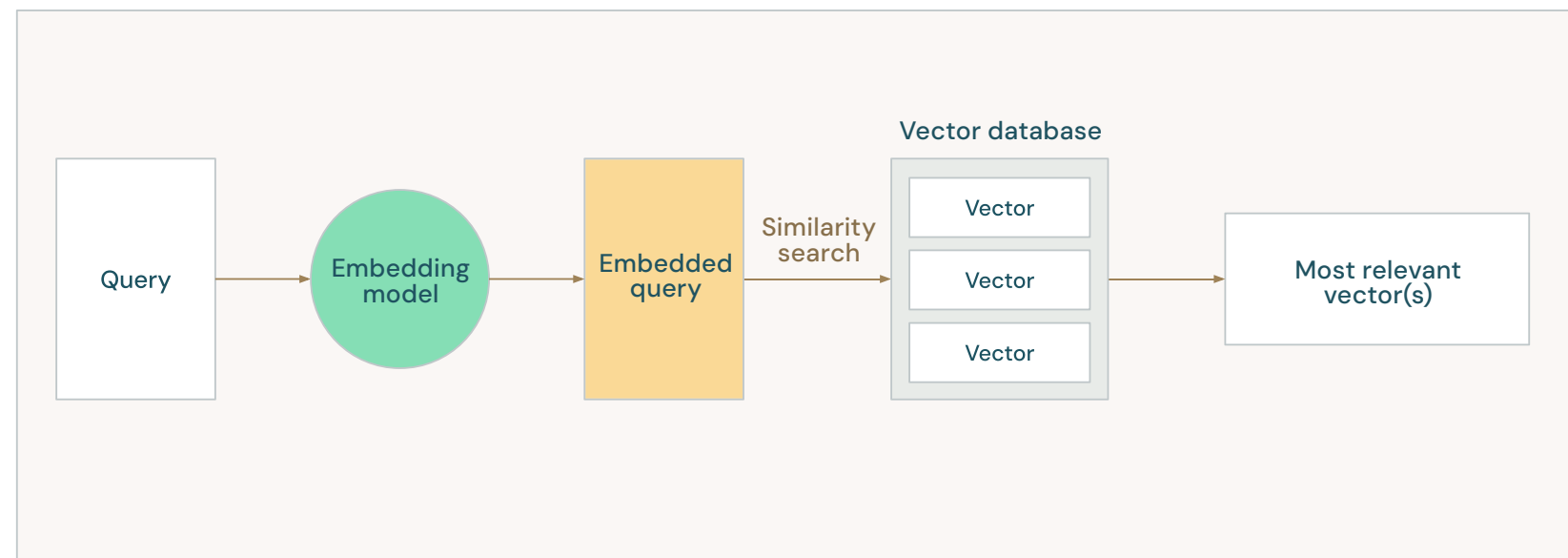
Chunking directly affects output quality. If chunks are too small, the retriever misses context — if they are too large, the model may struggle to pull out relevant details in the prompt window. The best chunking strategy depends on document structure and tasks and usually requires experimentation.

Retrieval

At query time, the system turns the user request plus system instructions into one or more retrieval queries. Retrieval must respect metadata filters (time ranges, document types, permissions) and typically combines:

- **Sparse retrieval:** Match exact tokens, IDs and rare terms
- **Dense retrieval:** Capture semantic similarity and paraphrases

Hybrid search — combining sparse and dense signals — is a practical default. It ensures that exact terms and IDs aren't lost while still benefiting from the ability of the embeddings to match paraphrases and related concepts.



Reranking

Raw retrieval often produces a candidate set where the correct answer is present but not near the top of the list. Reranking applies a stronger relevance model over the candidate set — often the top 50 results — to order them by task-specific relevance.

Reranking systems can be smaller LLMs, cross-encoders or specialized ranking models that consider the full query and document together.

In Databricks benchmarks, adding reranking on top of Vector Search improved recall@10 by 15 percentage points (from 74% to 89%), with low additional latency, directly improving answer quality without major infrastructure changes.

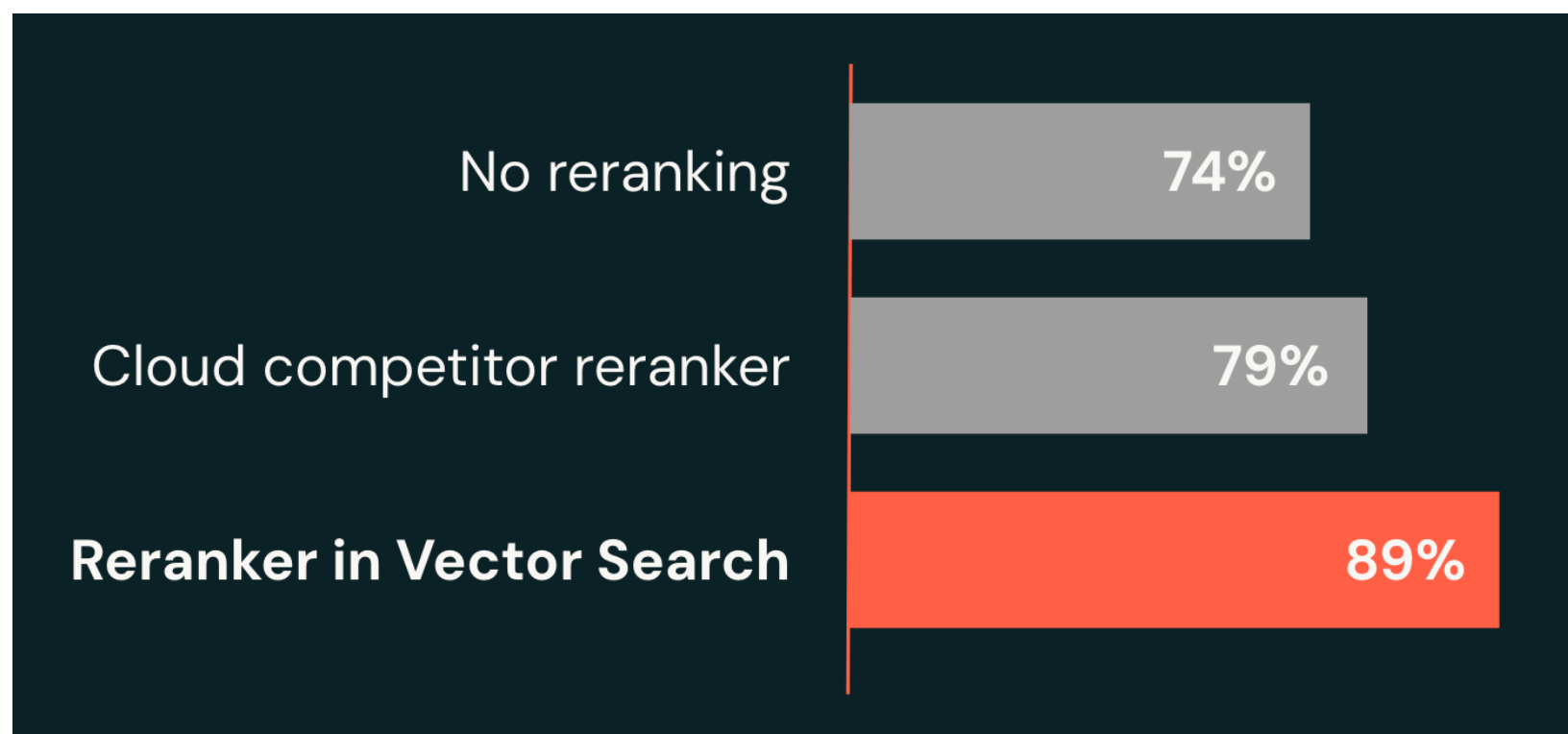


Figure: Enterprise benchmark showing recall@10 improvements with [Reranking in Vector Search](#).

Context assembly

Context assembly decides how to pack retrieved content into the model's input window, balancing:

- **Breadth:** How many documents to include
- **Depth:** How much of each document to include
- **Structure:** How to group or annotate snippets so the model can use them effectively

Simple strategies include concatenating top-k snippets with clear headings and citing their sources.

Advanced strategies cluster related chunks or group by document to preserve structure and reduce duplication.

Generation

Generation is where the LLM produces an answer using the assembled context, user query and system instructions.

System prompts should enforce grounding, such as:

- Answer using only the provided context where possible
- Cite or quote sources
- Indicate when the answer is uncertain or not covered in the context

Well-designed prompts, combined with good retrieval, help reduce hallucinations and make answers auditable.

Evaluation and monitoring

Evaluation and monitoring help determine if your RAG application meets your quality, cost and latency requirements.

- Evaluation occurs during development
- Monitoring happens once the application is deployed to production

RAG over unstructured data has many components that impact quality. For example, data formatting changes can influence the retrieved chunks and the LLM's ability to generate relevant responses. It's important to evaluate individual components in addition to the overall application.

RETRIEVAL METRICS

Retrieval metrics measure whether the retriever surfaces relevant content. Common choices include:

- **recall@k**: Probability that at least one relevant document is in the top-k results
- **MRR (mean reciprocal rank)**: How high the first relevant document appears on average
- **NDCG (normalized discounted cumulative gain)**: How good the ranking is when you've graded relevance

These metrics are computed against labeled datasets where each query has known relevant documents and are especially important when tuning hybrid weights, chunking strategies and reranking systems.

ANSWER METRICS

Answer metrics measure how good the final answers are. Useful signals include:

- **Exact match/F1**: How closely answers match references when there is a single correct answer
- **Semantic similarity**: How close answers are in meaning
- **Faithfulness/groundedness**: Whether answers are supported by retrieved context

LLM JUDGES FOR RAG EVALUATION

LLM judges provide automated evaluation of RAG responses by using a powerful LLM to assess answer quality along specific dimensions. This is particularly valuable for evaluating groundedness at scale.

RetrievalGroundedness judge

The RetrievalGroundedness judge assesses whether your application's response is factually supported by the provided context (either from a RAG system or generated by a tool call), helping detect hallucinations not backed by that context.

This built-in LLM judge is designed to evaluate RAG applications that require responses to be grounded in retrieved information. It examines each claim in the generated answer and verifies whether it can be traced back to the retrieved context.

Key benefits:

- Automated detection of hallucinations and unsupported claims
- Scalable evaluation across large test sets
- Consistent scoring criteria

LLM judges should be supplemented with spot checks for important workflows and used alongside retrieval metrics to provide a complete picture of RAG quality. Frameworks such as RAGAS can automate parts of this evaluation by scoring dimensions like faithfulness, answer relevance and context precision and recall.

Search traces by id, request, or response

Filters

Sort

Columns

Actions

Group by session

5 of

					Assessments (3/4)			Expectati
Trace ID	Request	Response	Execution t...	State	Correctness	Relevance	Retrieval groundedness	expectati
					PASS 100%	PASS 100%	PASS 100%	
					Pass 100% Fail 0%	Pass 100% Fail 0%	Pass 100% Fail 0%	
tr-9de5ad...	{ "query": "How long do...	"Matcha takes 28 days ...	8.241s	OK	Pass	Pass	Pass	Matcha fr
tr-e150bf...	{ "query": "What are the...	"BobaBricks offers almo...	12.55s	OK	Pass	Pass	Pass	Almond n
tr-42db18...	{ "query": "How do we p...	"For the Matcha Latte (...	8.621s	OK	Pass	Pass	Pass	Matcha r
tr-b534b1...	{ "query": "What is the ...	"The correct tapioca pe...	8.686s	OK	Pass	Pass	Pass	Cook 1 cu
tr-108efb...	{ "query": "How do I ma...	"To make a Brown Suga...	9.67s	OK	Pass	Pass	Pass	Brown Su

Figure: MLflow Evaluation UI to quickly identify root causes.

PRODUCTION MONITORING

Once deployed, continuously monitor your RAG application to catch quality degradation and operational issues:

- **Quality metrics:** Track answer correctness, groundedness scores and user feedback (thumbs up or down, explicit ratings)
- **Retrieval performance:** Monitor recall@k and retrieval latency to detect index staleness or configuration drift
- **Operational metrics:** Track end-to-end latency, token usage, costs and error rates
- **User behavior:** Analyze query patterns, follow-up questions and abandonment rates to identify gaps in coverage

Set up alerts for metric degradation and use production logs to build evaluation datasets that reflect real user needs.

DATASETS AND FEEDBACK

High-quality evaluation requires high-quality data. Good practices include:

- Building golden sets from production logs and customer feedback, with labeled relevant documents and reference answers
- Covering different tasks (short answers, lists, long-form reports) and different failure modes (missing docs, wrong docs, stale docs)
- Using these datasets in continuous integration to compare retrieval and answer metrics before and after changes

This turns evaluation into a feedback loop rather than a one-off exercise.

Current best practice: hybrid retrieval plus reranking

Across many deployments, a common pattern emerges for reliable RAG:

1. Use hybrid retrieval that combines sparse (keyword/BM25) and dense (vector) search
2. Retrieve a reasonably large candidate set (e.g., top 50)
3. Apply a reranker tuned for your tasks to pick the top-k context documents
4. Assemble context with clear structure and citations
5. Evaluate with retrieval and groundedness metrics
6. Iterate

Hybrid retrieval ensures that exact identifiers, keywords and rare terms are not lost, while dense search captures semantics and paraphrases. Reranking then promotes the most relevant candidates into the narrow band your model actually sees.

Databricks case studies show that this combination can significantly increase recall@k and answer quality compared to either sparse-only or dense-only approaches. See [Creating High-Quality RAG Applications With Databricks](#).

For many organizations, this architecture — plus good ingestion, chunking and evaluation — is the practical standard for high-quality RAG today.

Hybrid search implementation

Hybrid search enhances recall by adding a learned keyword index to the vector index. This index, trained on the dataset, captures critical keywords and identifiers essential for accurate retrieval.

Hybrid search is particularly effective when exact keyword matching is crucial, such as in cases involving product codes or technical terms.

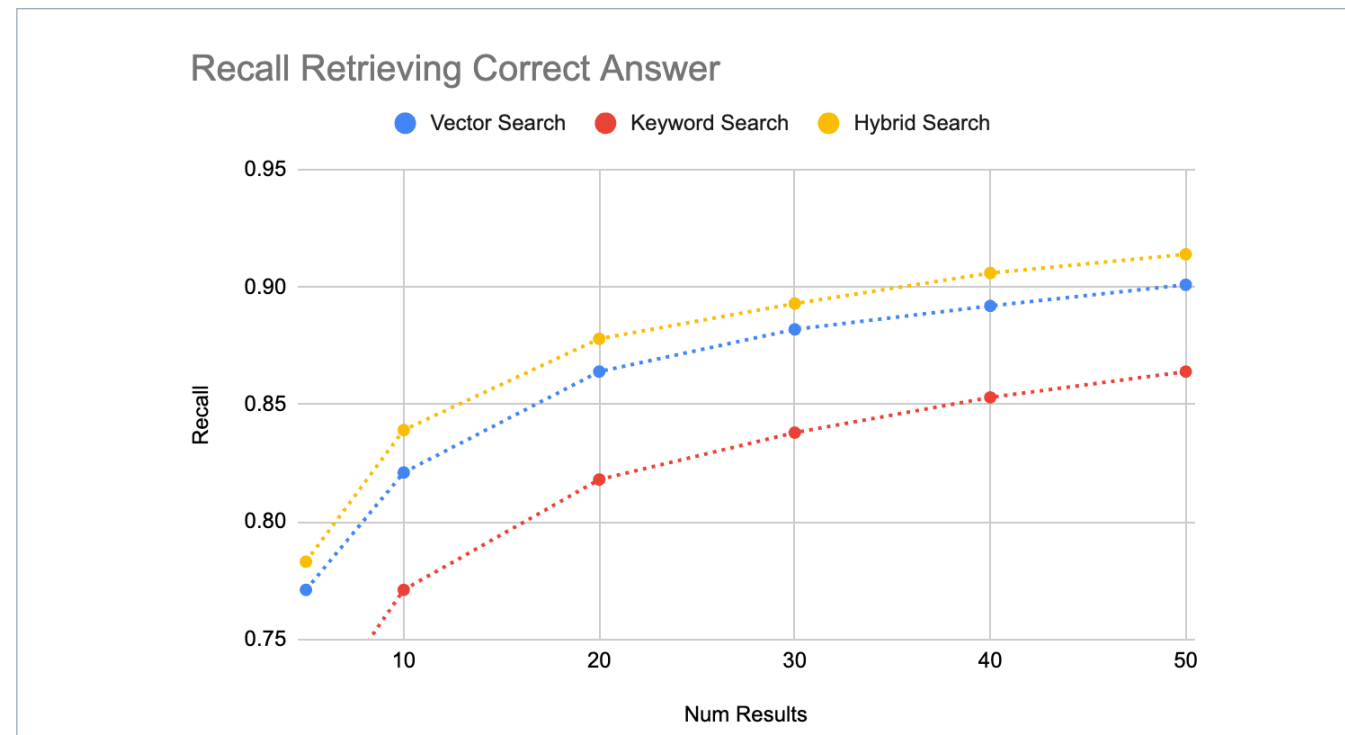
Hybrid search is straightforward to implement, as most indexing systems now support it without additional setup. By training the keyword index on all text fields in the corpus, both text chunks and metadata fields become searchable.

In RAG applications, a key performance metric is recall — the fraction of queries for which the correct chunk is retrieved in the top results. Hybrid search improves recall by reducing the number of chunks the LLM needs to process, thereby lowering latency and processing costs.

Internal benchmarks show a 20% improvement in recall, reducing the number of documents required for high recall from 50 to 40 in typical datasets.

IMPLEMENTATION DETAILS

Databricks implementation of hybrid search is based on [rank reciprocal fusion \(RRF\)](#) of the Vector Search and keyword search results. The RRF parameters are tuned to values that should return high-quality results for most datasets.



Scores are normalized so the highest score possible is 1.0:

- **Scores close to 1.0:** Both retrievers found the document to be of high relevance
- **Scores close to 0.5 and below:** One or both retrievers believe the document has low relevance

For more information, see the Databricks documentation:

- [Similarity search calculation details with hybrid search](#)
- [Python SDK for similarity_search](#)

Beyond basic RAG: Instruction-aware retrieval

This guide has focused on RAG done right: hybrid retrieval, reranking and retrieval-aware evaluation. These patterns are sufficient for most production assistants, copilots and internal search agents today.

In more complex enterprise agents, however, retrieval must follow detailed instructions and schemas: inclusion/exclusion rules, recency constraints, column-level filters and task-specific examples across multiple turns. Traditional RAG pipelines struggle here, even with strong embeddings and reranking systems.

The [Instructed Retriever](#) architecture from Databricks propagates full system specifications — user instructions, labeled examples and index schema — through query generation, retrieval, reranking and response.

This enables query decomposition into structured search plans, instruction-aware reranking (e.g., boosting recency) and metadata reasoning that turns natural language constraints into executable filters.

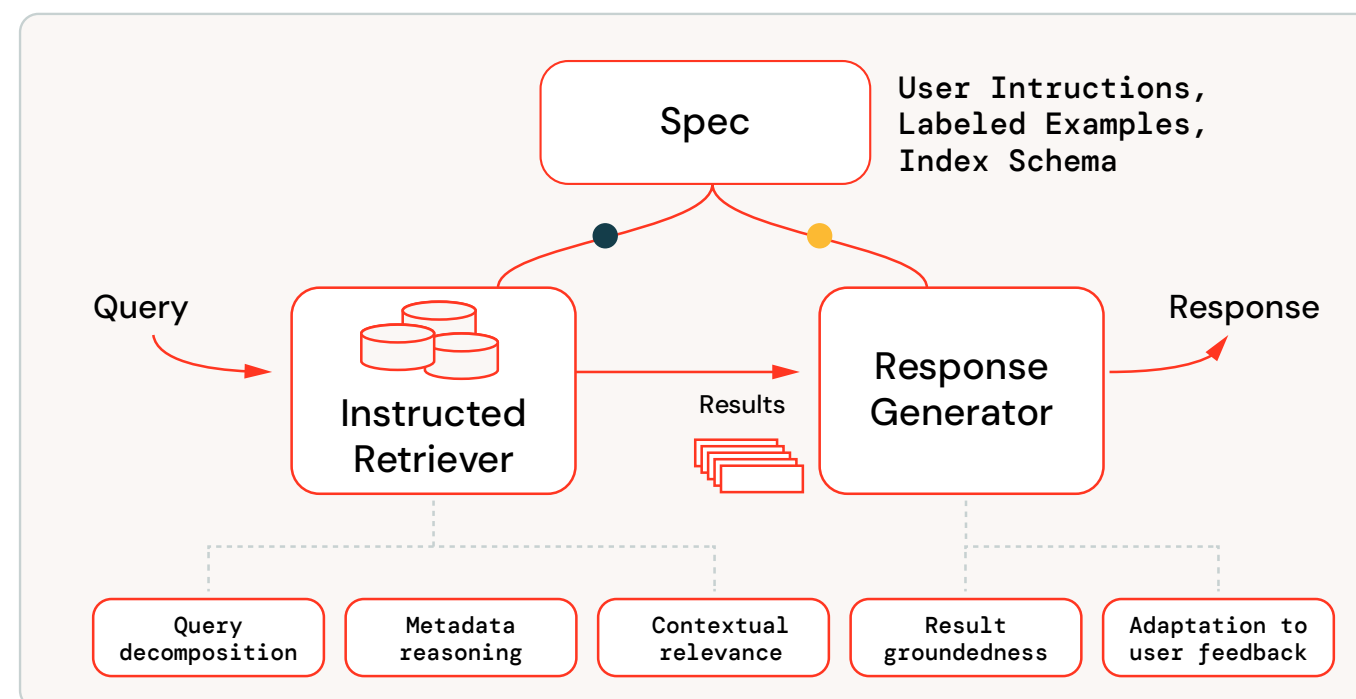


Figure: [Instructed Retriever](#) architecture from Databricks.

GRAPH RAG

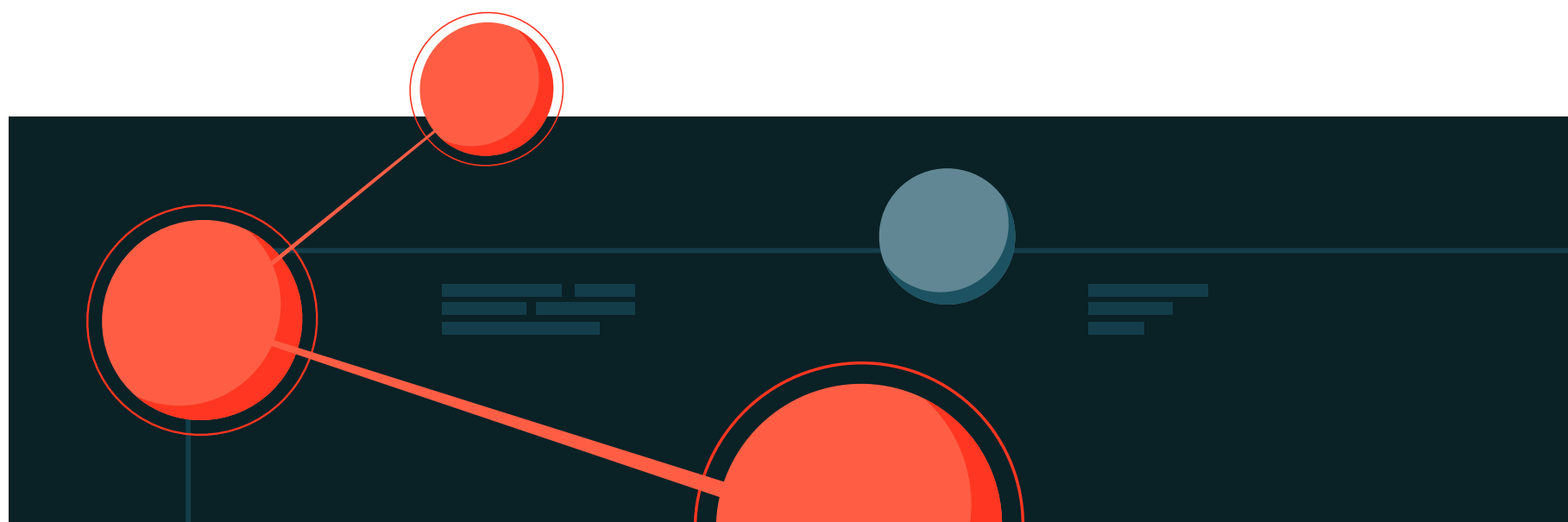
Graph RAG builds a knowledge graph from documents — extracting entities and relationships — and uses graph traversal during retrieval. This approach can surface multi-hop connections and relationship patterns that pure vector or keyword search might miss. This makes it appealing for highly interconnected domains like organizational charts, supply chains, citation networks or research papers where understanding entity relationships is crucial.

Graph RAG can help with:

- **Multi-hop reasoning:** Answering questions that require traversing several relationship links
- **Community detection:** Identifying clusters of related concepts or entities
- **Relationship-centric queries:** Finding connections between people, products or concepts

However, Graph RAG introduces significant complexity: entity extraction and graph maintenance overhead, additional infrastructure requirements and more complex retrieval logic. For most enterprise use cases — documentation search, policy Q&A, code search, ticket analysis — hybrid retrieval plus reranking delivers strong results with far less complexity.

Consider Graph RAG only when you have clear evidence that relationship traversal is essential and standard approaches fall short.



RAG on Databricks

While it's possible to set up a quick RAG demo in minutes, production-ready RAG requires careful orchestration of components in a reliable, scalable and secure manner. Databricks offers an end-to-end solution:

- **Databricks Vector Search:** Managed vector database with hybrid search, reranking and automatic updates
- **MLflow:** Track experiments, evaluate retrieval and answer quality, and manage model lifecycle
- **Model Serving:** Deploy RAG chains with low latency and automatic scaling
- **Unity Catalog:** Govern data access, maintain lineage and enforce permissions
- **Mosaic AI Agent Framework:** Build, evaluate and deploy agents using RAG alongside other tools

Agent Bricks: Automated agent development

Agent Bricks changes how enterprises develop AI agents powered by RAG. Instead of managing complexity, teams focus on defining their agents' purpose and providing quality feedback in natural language. Agent Bricks handles the rest:

1. **Declare your task:** Define what you want the agent to accomplish and connect data sources
2. **Automatic evaluation:** Agent Bricks creates task-specific evaluation benchmarks
3. **Automatic optimization:** Intelligently combines RAG, prompt engineering, fine-tuning and other optimization techniques automatically
4. **Cost and quality balance:** Choose between cost-optimized or quality-optimized agents, often achieving both

Agent Bricks is optimized for common use cases: structured information extraction, reliable knowledge assistance, custom text transformation and multi-agent systems. It provides one of the fastest ways to build AI agents that utilize RAG and is optimized for high-quality results.

Summary

Whether you're looking to disrupt traditional industries, enhance creative endeavors or solve complex problems in novel ways, the potential applications of generative AI and AI agents are limited only by your imagination and willingness to experiment. Remember, every significant advancement in this field began with a simple idea and the courage to explore it further.

For those seeking more knowledge or who are simply curious about the latest developments in the realm of generative AI and AI agents, we've provided some resources on training, demos and product information.

GenAI training

Generative AI Engineer Learning Pathway: Take self-paced, on-demand and instructor-led courses on generative AI

AI Agent Fundamentals: This foundational course introduces AI agents and their use in enterprise applications on Databricks

Get Started With AI Agents on Databricks: This course is an introduction to AI agents, their role in modern AI applications and how to build AI agent applications on the Databricks platform

Free LLM Course (edX): An in-depth course to learn GenAI and LLMs inside and out

GenAI Webinar: Learn how to take control of your GenAI app performance, privacy and cost, as well as drive value with GenAI

Additional resources

The Big Book of MLOps: A deep dive into the architectures and technologies behind MLOps — including LLMs and GenAI

The Big Book of GenAI: A deep dive into building GenAI applications

A Compact Guide to AI Agents: A concise guide to building AI agents

Agent Bricks: Product page covering the features of Agent Bricks within Databricks

See How to Build Production-Quality GenAI Applications

Create high-quality generative AI applications and ensure your output is accurate, governed and safe. See why over 20,000 organizations worldwide rely on Databricks for all their workloads from BI to AI — test-drive the full Databricks Platform free for 14 days.

[Try Databricks](#)[Take Generative AI Fundamentals On-Demand](#)

About Databricks

Databricks is the data and AI company. More than 20,000 organizations worldwide — adidas, AT&T, Bayer, Block, Mastercard, Rivian, Unilever and over 60% of the Fortune 500 — rely on Databricks to build and scale data and AI apps, analytics and agents. Headquartered in San Francisco with 30+ offices around the globe, Databricks offers a unified Data Intelligence Platform that includes Agent Bricks, Genie, Lakeflow, Lakehouse, Lakebase and Unity Catalog. To learn more, follow Databricks on [LinkedIn](#), [X](#), [YouTube](#) and [Instagram](#).

